

Programmation - Examen

Durée : 2 heures.

Les documents manuscrits et dactylographiés sont autorisés. La clarté et la précision de la rédaction seront un facteur important de l'évaluation. Le barème est donné à titre indicatif.

Exercice 1

```
1 public class Person {  
2     public String name, birthDate;  
3     ...  
4 }  
  
1 /**  
2  * Classe décrivant les personnes pratiquant des activités sportives  
3  * dans un club. Les clés sont les noms de sports, et les valeurs  
4  * associées les ensembles (non vides) des personnes qui pratiquent  
5  * ce sport dans ce club.  
6  * Exemple :  
7  * "football" : { ("Anne Pszczólkowski", "10/10/1980"),  
8  *                ("Mériadec Kermarrec", "01/02/2003") }  
9  * "curling" : { ("Michel Rafolomanantsiatosika", "12/11/1990") }  
10 */  
11 public class Club extends Table<String, Set<Person>>
```

```
1 /**  
2  * Classe décrivant un réseau de clubs de sport. Les clés sont les  
3  * noms des clubs, et les valeurs associées les personnes y  
4  * pratiquant ce sport (cette table peut être vide).  
5  */  
6 public class ClubNetwork extends Table<String, Club>
```

Les réponses seront écrites à l'aide des méthodes des classes *Table* et *Set* vues en TD.
Écrire les méthodes d'instance suivantes de la classe *ClubNetwork* :

Question 1 (2 pt) ✓

```
1 /**
2  * @return true si le sport sportName est pratiqué dans le club
3  * clubName, false sinon
4  */
5 public boolean practicedSport(String clubName, String sportName)
```

Question 2 (4 pt)

```
1 /**
2  * @return nombre de personnes pratiquant tous les sports proposés
3  * par le club clubName
4  */
5 public int countVeryAthleticMembers(String clubName)
```

Question 3 (4 pt)

```
1 /**
2  * Désinscrire person de toutes les activités des clubs du réseau.
3  * @param person personne à désinscrire
4  */
5 public void removePerson(Person person)
```

Question 4 (4 pt)

```
1 /**
2  * Le club clubName2 est racheté par clubName1. Les activités du
3  * clubName2 sont donc intégrées dans clubName1 et clubName2 est
4  * supprimé du réseau.
5  * @param clubName1 nom du club acheteur
6  * @param clubName2 nom du club acheté
7  * @pre this.contains(clubName1) && this.contains(clubName2)
8  */
9 public void acquisition(String clubName1, String clubName2)
```

Question 5 (6 pt)

```
1 /**
2  * @return le sport qui, en considérant tous les clubs, est pratiqué
3  * par le plus grand nombre de personnes.
4  */
5 public String mostPopularSport()
```


Correction Examen Final sur les
Ensembles et tableaux associatifs

Exercice 1

Question 1:

```
public boolean practicedSport(String clubName, String sportName){  
    Club myClub = this.getValue(clubName);  
    if (myClub != null){  
        Set  
        return myClub.contains(sportName);  
    } else {  
        return false;  
    }  
}
```

```
public int countVeryAthleticMembers(String clubName){  
    int cpt = 0;  
    Club myClub = this.getValue(clubName);  
    Iterator<String> it = myClub.iterator();  
    while (it.hasNext()) {  
        String key = it.next();  
        Set<Person> mySet = myClub.getValue(key);  
        cpt = mySet.size() + cpt;  
    }  
    return cpt;  
}
```


Question 3.

```
public void removePerson(Person person){
    Iterator<String> it1 = this.iterator();
    while(it1.hasNext()){
        String Key1 = it1.next();
        Club myClub = this.getValue(Key1);
        Iterator<String> it2 = myClub.iterator();
        while(it2.hasNext()){
            String Key2 = it2.next();
            Set<Person> mySet = myClub.getValue(Key2);
            mySet.remove(person);
            if(mySet.isEmpty()){
                it2.remove();
            }
        }
        if(myClub.isEmpty()){
            it1.remove();
        }
    }
}
```


Question 4.

```
public void acquisition (String clubName1, String clubName2) {  
    Club myClub1 = this.getValue(clubName1);  
    Club myClub2 = this.getValue(clubName2);  
    Iterator<String> it2 = this myClub2.iterator();  
    while (it2.hasNext()) {  
        String key2 = myClub2 it2.next();  
        Set<Person> mySet1 = myClub1.getValue(key2);  
        Set<Person> mySet2 = myClub2.getValue(key2);  
        if (mySet1 != null) {  
            mySet1.union(mySet2);  
        } else {  
            myClub1.addValue(key2, mySet2);  
        }  
    }  
    this.removeValue(clubName2);  
}
```


Question 5

```
public String mostPopularSport() {
```

```
    Table<String, Set<Person>> sportAndPersonAll = new Table<>();
```

```
    Iterator it = this.iterator();
```

```
    while(it.hasNext()) {
```

```
        String key = it.next();
```

```
        Club myClub = this.getValue(key);
```

```
        Iterator<String> it2 = myClub.iterator();
```

```
        while(it2.hasNext()) {
```

```
            String key2 = it2.next();
```

```
            Set<Person> mySet mySetClub = myClub.getValue(key);
```

```
            Set<Person> mySetSport = sportAndPersonAll.getValue(key);
```

```
            if(mySetSport != null) {
```

```
                mySetSport.union(mySetClub);
```

```
            } else {
```

```
                sportAndPersonAll.addValue(key2, mySetClub);
```

```
            }
```

```
        }
```

```
    }
```

```
int cph = 0;
```

```
int cpe = 0;
```

```
String sport
```

64


```

String sportNameMost;
int cpt = 0;
Iterator<String> it3 = sportAndPersonAll.iterator();
while(it3.hasNext()) {
    String key3 = it3.next(); // etape 4 nom du sport
    Set<Person> mySet = sportAndPersonAll.getValue(key3);
    if (cpt < mySet.size()) {
        cpt = mySet.size();
        sportNameMost = key3;
    }
}

return sportNameMost;
}

```

CASE DOUBLE

Question 2:

```
public int countVeryAthleticMembers(String clubName){  
    boolean bool = true; // pour vérifier si c'est le 1er tour  
    Club myClub = this.getValue(clubName);  
    Set<Person> mySetPratiquantAll = new HashSet<>(); // Pour récupérer  
                                                    les personnes  
                                                    qui pratiquent  
                                                    tous les sports  
                                                    en même temps  
  
    Iterator<String> it = myClub.iterator();  
    while (it.hasNext()) {  
        String key = it.next();  
        Set<Person> mySetClub = myClub.getValue(key);  
        if (bool) {  
            mySetPratiquantAll.union(mySetClub); // car le mySetPratiquantAll  
                                                    est vide au départ  
            bool = false;  
        } else {  
            mySetPratiquantAll.intersection(mySetClub);  
        }  
    }  
    return mySetPratiquantAll.size();  
}
```